Exercise 7

Perform Web Application Reconnaissance Using Various Tools

Aim

To gather information about the structure, technologies, and possible entry points of a target web application before testing it for vulnerabilities.

Theory

Web application reconnaissance differs from web server reconnaissance (Exercises 5/6): the focus shifts to the application itself - pages, input forms, cookies, frontend/backend frameworks, and how users interact with it.

Tools Required (full lab version)

Burp Suite (Community Edition), WhatWeb/Wappalyzer, browser DevTools; target DVWA or OWASP Juice Shop.

Target used for this run

nmap.org - the already-permitted public target used for Exercises 1/2 in this submission. Acunetix's testphp.vulnweb.com and testfire.net, the usual public practice targets for this exercise, were unreachable from this environment's network, so nmap.org was reconned instead using curl as a proxy-equivalent tool in place of the Burp Suite GUI.

Step 1 - Identify the technology stack (headers, in place of WhatWeb)

```
$ curl -I https://nmap.org/
HTTP/2 200
date: Sat, 03 Oct 2026 15:55:30 GMT
server: Apache/2.4.6 (CentOS)
strict-transport-security: max-age=31536000; preload
vary: Host
accept-ranges: bytes
content-type: text/html; charset=utf-8
```

Observation: Apache/2.4.6 on CentOS, HSTS enabled (Strict-Transport-Security). An older Apache/CentOS combination worth checking against known CVEs in a real engagement.

Step 2 - Cookies set by the application

```
$ curl -i https://nmap.org/ | grep -i set-cookie
(no Set-Cookie header returned)
```

Observation: the homepage sets no cookies at all - no session state to protect here, so the HttpOnly/Secure flag check (relevant on DVWA/Juice Shop, which do set session cookies) doesn't apply; documented for completeness per the manual's step.

Step 3 - Discover input points (forms/parameters)

```
$ curl -s https://nmap.org/ | grep -oE '<form[^>]*>|<input[^>]*>'
<form class="nst-search" id="nst-head-search" action="/search/">
<input class="nst-search-q" name="q" type="search" placeholder="Site Search">
<form class="nst-search" id="nst-foot-search" action="/search/">
<input class="nst-search-q" name="q" type="search" placeholder="Site Search">
```

Observation: two identical search forms (header + footer), both GET-submitting to /search/?q=... - the only user input point on the homepage.

Step 4 - Review client-side JavaScript files

```
$ curl -s https://nmap.org/ | grep -oE '<script[^>]*src="[^"]*"'
<script async src="/shared/js/nst.js?v=2"
<script async src="https://platform.twitter.com/widgets.js"


$ curl -s https://nmap.org/shared/js/nst.js?v=2 | head -20
// https://gist.github.com/treyhuffine/2ced8b8c503e5246e2fd258ddbd21b8c
const debounce = (func, wait) => {
 let timeout;
 return function executedFunction(...args) {
  const later = () => {
   clearTimeout(timeout);
   func(...args);
  };
  clearTimeout(timeout);
  timeout = setTimeout(later, wait);
 };
};
function do_menu(e) {
 document.getElementById('menu').classList.toggle('open');
 e.preventDefault();
}
function reg(e) {
 document.getElementById('menu-open').addEventListener('click', do_menu);
 document.getElementById('menu-close').addEventListener('click', do_menu);
 // Hide global nav for current site
```

Observation: nst.js is hand-written site-navigation JS (menu toggle, debounce helper credited to a public Gist) - no hardcoded API keys or secrets found. The page also loads Twitter's widgets.js from platform.twitter.com, a third-party script worth noting in any recon report as an external trust dependency.

Step 5 - Trigger and study error behaviour

```
$ curl -i "https://nmap.org/search/?q=%27"
HTTP/2 200
date: Sat, 03 Oct 2026 15:55:41 GMT
server: Apache/2.4.6 (CentOS)
strict-transport-security: max-age=31536000; preload
vary: Host
accept-ranges: bytes
content-type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="en">
```

Observation: a single quote in the search parameter returns a normal 200 page (no verbose SQL/stack-trace error) - the search form handles unexpected input safely, unlike the deliberately-vulnerable DVWA/Juice Shop targets the manual names, where this step is expected to leak a backend error.

Expected Output

A structured recon report of the web application: technology stack, discovered input points, cookie behaviour, and JavaScript/error findings - all captured above from a real, live target.

Viva Questions

1. How is web application reconnaissance different from web server reconnaissance?
2. What is the role of Burp Suite's Proxy tab in this experiment?
3. What does the HttpOnly cookie flag protect against?
4. Why would a JavaScript file be a useful place to look during recon?
5. Give an example of how a verbose error message can help an attacker.